Week 5, Day 2: S3, RDS, ElastiCache, SQS, SNS and Topological Sort

Prepared for Upendra Kumar Sharma | 4 hours, plus an optional 90-minute lab | Main topic: the AWS data and messaging services your backend uses most | DSA: Course Schedule and Course Schedule II (topological sort)

Today's Schedule

- **0:00 to 0:45** Study S3 (section 1).
- **0:45 to 1:30** Study RDS and Aurora (section 2).
- **1:30 to 1:50** Study ElastiCache (section 3).
- **1:50 to 2:40** Study SQS, SNS and how they compare with Kafka (sections 4 and 5).
- **2:40 to 3:30** DSA: solve the 2 problems in section 7. Allow 25 minutes each.
- **3:30 to 4:00** Answer the interview questions aloud (section 8) and add 10 flashcards.
- **Optional (90 min):** the hands-on lab in section 6. RDS and ElastiCache are the costly parts, so create them, use them, and delete them the same day.

Keep your billing alert on. Delete every resource you create when you finish.

1. Amazon S3 (Simple Storage Service)

What it is: object storage. You store **objects** (a file plus metadata) in **buckets**. A bucket name is **globally unique**, and the bucket lives in one region. The namespace is flat: a key such as reports/2026/01/a.pdf is just a string, and folders in the console are only a display convention using prefixes.

Key facts

- **Durability** is designed for 99.999999999 percent (eleven nines) by storing data redundantly across several AZs. Availability depends on the storage class.
- **Consistency:** strong read-after-write consistency for puts, overwrites and deletes. You read your own writes.
- **Size:** one object up to 5 TB. A single PUT is limited to 5 GB, so use **multipart upload** for large files (recommended from about 100 MB), which also allows parallel and resumable uploads.
- **Cost:** you pay for storage, requests and **data transferred out** to the internet. Uploading data in is free.

Storage classes (choose by access pattern)

- **Standard:** frequent access.
- **Intelligent-Tiering:** moves objects between tiers automatically when access patterns are unknown or change.
- **Standard-IA and One Zone-IA:** infrequent access, cheaper storage with a retrieval fee (One Zone-IA keeps data in a single AZ, so it is less resilient).
- **Glacier Instant Retrieval, Flexible Retrieval and Deep Archive:** archival, very cheap, with retrieval times from milliseconds to hours.
- **Lifecycle rules** move objects between classes and delete them after a set time, which is the standard way to control cost and apply retention.

Security (interviewers ask about this a lot)

- **Block Public Access** is on by default. Keep it on unless you truly serve public content, and then put CloudFront in front.
- **Access control:** IAM policies for your identities, **bucket policies** for rules on the bucket (for example deny any request not using TLS, or allow one role), and ACLs are disabled by default in new buckets. Apply least privilege and scope policies to prefixes.
- **Encryption at rest:** SSE-S3 (the default), **SSE-KMS** (your own key, with an audit trail and key policies) or client-side encryption. **Encryption in transit:** HTTPS.
- **Presigned URLs:** a time-limited URL that lets a client download or upload **one specific object** without AWS credentials. Your API checks authorization, generates the URL, and the client talks to S3 directly, so large files never pass through your servers.
- **VPC gateway endpoint** for S3: private, free access from your VPC without a NAT gateway.

Features worth naming

- **Versioning** keeps old versions and protects against accidental deletion or overwriting (a delete adds a marker). Pair it with lifecycle rules to expire old versions.
- **Replication** (same-region or cross-region) for compliance and disaster recovery.
- **Event notifications** on object creation or deletion, sent to SQS, SNS, Lambda or EventBridge. Used for virus scanning, thumbnails and processing pipelines.
- **Object Lock** for write-once, read-many retention (compliance needs).
- **Static website hosting**, usually with CloudFront in front.

In your complaint or order domain: store documents in S3 and **only the metadata (key, size, checksum, owner) in the database**. Use presigned URLs for upload and download, SSE-KMS, versioning, and a lifecycle rule for retention. An upload event can trigger a malware scan.

Using it from Java (AWS SDK v2)

```
@Service
public class DocumentStorage {
    private final S3Client s3;
    private final S3Presigner presigner;
    private final String bucket;
    // constructor omitted: credentials come from the default provider chain
(the instance or task role)

    public void upload(String key, byte[] content, String contentType) {

s3.putObject(PutObjectRequest.builder().bucket(bucket).key(key).contentType(c
ontentType).build(),
                RequestBody.fromBytes(content));
    }

    public String downloadUrl(String key) {
        GetObjectRequest get =
GetObjectRequest.builder().bucket(bucket).key(key).build();
        GetObjectPresignRequest request = GetObjectPresignRequest.builder()
                .signatureDuration(Duration.ofMinutes(10))
                .getObjectRequest(get)
                .build();
        return presigner.presignGetObject(request).url().toString();
    }
}
```

Spring Cloud AWS (version 3 for Spring Boot 3) adds higher-level helpers such as S3Template, but know the plain SDK first.

2. Amazon RDS and Aurora

What RDS is: a managed relational database service for PostgreSQL, MySQL, MariaDB, Oracle, SQL Server and Db2. AWS provisions the instance, patches the engine, runs backups and handles failover. You manage the schema, queries, parameters and access. You do not get operating system access or true superuser rights.

What it gives you

- **Automated backups** (daily snapshot plus transaction logs) with **point-in-time restore** within a retention window of up to 35 days. **Manual snapshots** are kept until you delete them. Test a restore. A backup you have never restored is not proven.
- **Multi-AZ:** a **synchronous standby in another AZ** with **automatic failover** (usually one to two minutes) when the primary fails. It is for **high availability**, not for read scaling, because in the classic instance form the standby cannot serve reads. It roughly

doubles the instance cost.

- **Read replicas: asynchronous** copies used to **scale reads** (and for cross-region reads or disaster recovery). They can lag behind the primary, so a read right after a write may be stale. They can be promoted to standalone databases.
- **Storage:** gp3 or provisioned IOPS, with storage autoscaling.
- **Security:** place it in **private subnets** (a DB subnet group spanning at least two AZs), allow access only from the **application security group**, encrypt at rest with KMS and in transit with TLS, and keep credentials in **Secrets Manager** (with automatic rotation) or use IAM database authentication.
- **Operations:** parameter groups (engine settings), maintenance windows, **Performance Insights** and CloudWatch metrics, and **RDS Proxy** (a managed connection pool that helps with many short-lived connections such as Lambda, and shortens failover for applications).

Aurora (PostgreSQL and MySQL compatible)

- A cloud-built engine with storage shared across **three AZs** (six copies), automatic growth, up to **15 low-lag read replicas** that share the storage, and faster failover than classic Multi-AZ.
- **Aurora Serverless v2** scales capacity up and down with load.
- It costs more than plain RDS in many cases, and suits workloads that need higher read scale, faster failover or variable load.

What your application must do

- **Survive failover:** the endpoint stays the same, but it points to a new instance after failover. Existing connections break, so use a connection pool with validation (HikariCP), sensible **maxLifetime and connection timeouts**, and retries for idempotent operations. Check the JVM DNS cache setting (**networkaddress.cache.ttl**), so the application notices the new address instead of holding the old one.
- **Use a connection string with the instance endpoint** from configuration, and credentials from Secrets Manager or Parameter Store, never hard-coded.
- **Run schema changes through Flyway** at deployment time, in a controlled way.
- **Do not exhaust connections:** total pool sizes across all instances must fit the database's maximum connections.

RDS vs running PostgreSQL yourself on EC2: RDS removes the work of backups, patching and failover, and gives up some control. For most teams, RDS or Aurora is the right default.

3. Amazon ElastiCache

- **Managed Redis (and Valkey) or Memcached.** Lives **inside your VPC**, in private subnets, and is never exposed to the internet.
- **Redis replication group:** one primary and up to five replicas across AZs, with **automatic failover**. Use the **primary endpoint** for writes and the **reader endpoint** for reads.
- **Cluster mode enabled** shards data across several primaries, which is the way to scale memory and write throughput. Clients must support cluster mode, and multi-key operations need the keys in the same slot (hash tags, from Week 4).
- **Security:** security group allowing 6379 only from the application, **encryption in transit (TLS)** and at rest, and authentication (AUTH or role-based access control).
- **Configuration:** node types (for example cache.t4g and cache.r6g), parameter groups (maxmemory-policy, so choose allkeys-lru for a cache), snapshots, and metrics in CloudWatch (hit rate, evictions, memory, CPU, replication lag).
- **Redis vs Memcached:** Redis has data structures, persistence, replication and Lua scripting. Memcached is a simple multi-threaded key-value cache with no replication. For most Java backends, choose Redis.
- **Spring Boot:** set spring.data.redis.host to the endpoint, turn on TLS (spring.data.redis.ssl.enabled), keep short timeouts, and **keep the fallback behavior** from Week 4 (the application must work if the cache is down).
- **MemoryDB** is a Redis-compatible database with durable storage across AZs, for when Redis is the primary data store.

4. Amazon SQS (Simple Queue Service)

What it is: a fully managed message queue. Producers send messages, and **competing consumers** poll and process them. You manage no servers, and it scales automatically.

Standard vs FIFO

- **Standard queue:** very high throughput, **at-least-once** delivery (duplicates are possible), and **best-effort ordering**.
- **FIFO queue: strict ordering within a message group** (set a **MessageGroupId**, similar to a Kafka key) and **deduplication** over a short window (a **MessageDeduplicationId**, or content-based). It has lower throughput limits than standard (check the current quotas, including the high-throughput mode).

Concepts you must know

- **Visibility timeout (30 seconds by default):** when a consumer receives a message, it becomes **invisible** to others for this period. If the consumer **deletes** the message before it ends, the work is done. If not, the message **reappears** and is delivered again. Set it **longer than your processing time**, and extend it with ChangeMessageVisibility for long jobs. This is the main source of duplicates.
- **Delete after success:** the consumer deletes the message only after processing finishes. A crash simply leads to redelivery, which is at-least-once. **So consumers must be idempotent** (the inbox pattern from Week 3).
- **Long polling:** set WaitTimeSeconds up to 20, so that a receive call waits for messages instead of returning empty. It cuts cost and latency compared with short polling.
- **Batching:** send, receive and delete up to 10 messages per call.
- **Retention:** 4 days by default, up to 14 days. **Message size** has been limited to 256 KB, and recently larger sizes have been introduced, so check the current quota. For large payloads, store the data in S3 and send a **reference** (the claim check pattern).
- **Delay queues and message timers** postpone delivery.
- **Dead letter queue (DLQ):** configure a **redrive policy** with **maxReceiveCount**. After that many failed receives, SQS moves the message to the DLQ. Alarm on DLQ depth, and use **DLQ redrive** to send messages back after a fix.
- **Scaling consumers:** add more workers (for example, ECS tasks or Lambda scale on queue depth). There are no consumer groups or partitions, and with standard queues no ordering to preserve.

In Spring Boot: use **Spring Cloud AWS** (version 3 supports Spring Boot 3), where an @SqsListener method receives messages, and a normal return acknowledges (deletes) the message while an exception leaves it for redelivery. Or use the AWS SDK directly.

```
@Component
public class OrderCreatedConsumer {

    @SqsListener("inventory-order-created")
    public void handle(OrderCreated event) {
        reservationService.handle(event);    // idempotent; throw to let SQS
retry and eventually dead-letter
    }
}
```

5. Amazon SNS and the Fan-Out Pattern

What it is: a **publish and subscribe** service. A publisher sends to a **topic**, and SNS delivers a copy to every subscriber: SQS queues, Lambda functions, HTTP endpoints, email, SMS and mobile push. It has **standard** and **FIFO** topics.

- **Message filtering:** a **subscription filter policy** (on message attributes or the payload) lets each subscriber receive only the messages it cares about.
- **Delivery retries and a DLQ per subscription** protect against failing subscribers.
- **Raw message delivery** to SQS avoids the SNS envelope.
- **SQS must allow SNS** to send to it (a queue access policy).

The fan-out pattern (the most common design): publish an event once to an **SNS topic**, and subscribe **one SQS queue per consuming service**. Each service reads its own queue at its own pace, retries independently, has its own DLQ, and scales separately. This gives durability and independence, and one slow or broken consumer does not affect the others.

Related services to name: EventBridge (event bus with rich routing rules, schema registry and SaaS integrations, good for routing events between many services and accounts), and **Kinesis Data Streams** (an ordered, replayable stream, closer to Kafka).

Mapping your Kafka design to AWS-native services

- **Topic (orders.created):** an SNS topic.
- **Consumer group per service:** an SQS queue per service, subscribed to the topic.
- **Dead letter topic:** an SQS DLQ per queue.
- **Message key and ordering:** FIFO topic and queue with a MessageGroupId (accepting lower throughput).
- **Replay:** not available in SQS. If you need replay, archive events to S3, or use Kinesis or Amazon MSK (managed Kafka).
- **Outbox relay:** publishes to SNS instead of Kafka. The outbox, inbox and idempotency logic stay exactly the same, which is a good point to make: the **patterns are portable and only the transport changes**.

SQS vs Kafka (a classic question)

- **SQS:** simple, fully managed, per-message acknowledgement and retries, scales automatically, pay per request, **no replay and no ordering across the queue** (except FIFO groups).
- **Kafka:** a durable replayable log, ordering per partition, many independent consumer groups, very high throughput, more to operate (or costlier when managed).
- **Pick SQS** for task queues and decoupling with minimal operations. **Pick Kafka** for event streaming, replay, multiple independent consumers of the same stream and stream processing.

6. Optional Hands-On Lab (90 Minutes)

1. **S3.** Create a bucket with Block Public Access on and default encryption. Upload a file with the CLI, turn on versioning, overwrite the file and list the versions. Add a lifecycle rule. Generate a **presigned URL** with aws s3 presign (set a short expiry), open it, then try it again after it expires. Add a bucket policy that denies requests not using TLS.
2. **S3 from Java.** Add the upload and presigned download code to a small endpoint of your project, running on your EC2 instance with its role and no keys.
3. **RDS.** Create a small single-AZ PostgreSQL instance (a free-tier eligible class if available) in the private data subnets, with a security group that allows 5432 only from your application security group. Connect from your EC2 instance with psql, and run your Flyway migrations against it. Take a snapshot. **Delete the instance when you finish.**
4. **ElastiCache (optional, costs money).** Create the smallest Redis node, connect from EC2 with TLS using redis-cli, set and read a key, then delete it the same day.
5. **SQS.** Create a standard queue with a DLQ (maxReceiveCount 3). Send a message, receive it with a wait time of 10 seconds, **do not delete it**, and watch it reappear after the visibility timeout. Fail it three times and see it arrive in the DLQ. Then create a FIFO queue and send duplicates to see deduplication.
6. **SNS fan-out.** Create a topic and two queues subscribed to it, one with a **filter policy**. Publish messages with attributes, and see which queue receives what.
7. **Clean up** every resource, and check the billing page the next day.

7. DSA: Topological Sort (2 Problems)

What it is: an ordering of the nodes of a **directed acyclic graph (DAG)** so that every edge goes from earlier to later. It exists **only if there is no cycle**, so it also detects cycles. Typical uses: **course prerequisites, build and dependency order** (Maven modules, Spring bean creation, database migrations) and **task scheduling**.

Kahn's algorithm (BFS with in-degrees)

1. Build the adjacency list and count each node's **in-degree** (the number of prerequisites).
2. Put all nodes with in-degree 0 in a queue.
3. Remove a node from the queue, add it to the order, and decrease the in-degree of its neighbors. A neighbor that reaches 0 joins the queue.
4. If the order contains **all nodes**, it is valid. If fewer, a **cycle** blocked the rest.

DFS alternative: mark nodes as unvisited, in progress and done. Meeting an in-progress node means a cycle. Add a node to the order after its descendants finish, then reverse the order.

Problem 1: Course Schedule

- **The problem:** n courses, and prerequisite pairs [a, b] meaning you must take b before a. Can you finish all courses?
- **Idea:** build the graph with an edge from b to a, run Kahn's algorithm, and check that all n courses were taken. The question is just "is there a cycle?".
- **Complexity:** O(V + E) time and space.
- **Edge cases:** no prerequisites, a course that requires itself ([1, 1], which is a cycle), duplicate pairs, disconnected courses.

```
public boolean canFinish(int numCourses, int[][] prerequisites) {
    List<List<Integer>> graph = new ArrayList<>();
    for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());
    int[] indegree = new int[numCourses];
    for (int[] p : prerequisites) {
        graph.get(p[1]).add(p[0]);           // p[1] must come before p[0]
        indegree[p[0]]++;
    }

    Queue<Integer> queue = new ArrayDeque<>();
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0) queue.offer(i);
    }

    int taken = 0;
    while (!queue.isEmpty()) {
        int course = queue.poll();
        taken++;
        for (int next : graph.get(course)) {
            if (--indegree[next] == 0) queue.offer(next);
        }
    }
    return taken == numCourses;
}
```

Problem 2: Course Schedule II

- **The problem:** the same input, but return **one valid order** of the courses, or an empty array if it is impossible.

- **Idea:** the same algorithm, recording the order in which courses leave the queue. If the order is shorter than n, return an empty array.
- **Complexity:** O(V + E).
- **Note:** several valid orders usually exist, so tests accept any valid one. Be ready to explain that the answer is not unique.

```
public int[] findOrder(int numCourses, int[][] prerequisites) {
    List<List<Integer>> graph = new ArrayList<>();
    for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());
    int[] indegree = new int[numCourses];
    for (int[] p : prerequisites) {
        graph.get(p[1]).add(p[0]);
        indegree[p[0]]++;
    }

    Queue<Integer> queue = new ArrayDeque<>();
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0) queue.offer(i);
    }

    int[] order = new int[numCourses];
    int index = 0;
    while (!queue.isEmpty()) {
        int course = queue.poll();
        order[index++] = course;
        for (int next : graph.get(course)) {
            if (--indegree[next] == 0) queue.offer(next);
        }
    }
    return index == numCourses ? order : new int[0];
}
```

Pattern to remember: whenever a problem says "X must happen before Y", think of a directed graph and topological sort. Related problems: Alien Dictionary, Minimum Height Trees and Parallel Courses.

8. Interview Questions to Answer Aloud

1. What is S3, and how does it differ from EBS and EFS? What are its consistency and durability guarantees?
2. How do S3 storage classes and lifecycle rules reduce cost?
3. How would you let a browser upload a large file securely without sending it through your servers?

4. How do you secure an S3 bucket? Name at least five controls.
5. RDS Multi-AZ vs read replicas: purpose, replication type and effect on failover.
6. What does your application have to do to survive an RDS failover?
7. RDS vs Aurora: when would you choose each? What does RDS Proxy do?
8. How do you manage database credentials on AWS without hard-coding them?
9. How does ElastiCache for Redis provide high availability and scale? What are the primary and reader endpoints?
10. SQS standard vs FIFO: ordering, duplicates and throughput. What is a MessageGroupId?
11. Explain the visibility timeout. What happens if your processing takes longer than it?
12. What is a dead letter queue, and how do you operate one?
13. Explain SNS and the SNS to SQS fan-out pattern, and why it beats direct calls.
14. SQS vs Kafka: how do you choose? How would you port your Kafka-based project to SNS and SQS?

9. End-of-Day Checklist

- [ ] I can explain S3 classes, lifecycle, security controls, presigned URLs and event notifications.
- [ ] I can explain RDS backups, Multi-AZ vs read replicas, Aurora, and what an application must do on failover.
- [ ] I can explain ElastiCache replication groups, endpoints and cluster mode.
- [ ] I can explain SQS standard vs FIFO, the visibility timeout, long polling and DLQs.
- [ ] I can explain SNS fan-out and map my Kafka design to SNS and SQS, saying what is lost (replay).
- [ ] I solved Course Schedule and Course Schedule II and can explain Kahn's algorithm and cycle detection.
- [ ] (Optional) I completed the lab and deleted every resource.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Week 5, Day 3): Lambda, ECS and EKS overview, load balancers and CloudWatch, plus Rotting Oranges and Pacific Atlantic Water Flow.